

Retrieval Augmented Generation

Иноземцева Ксения

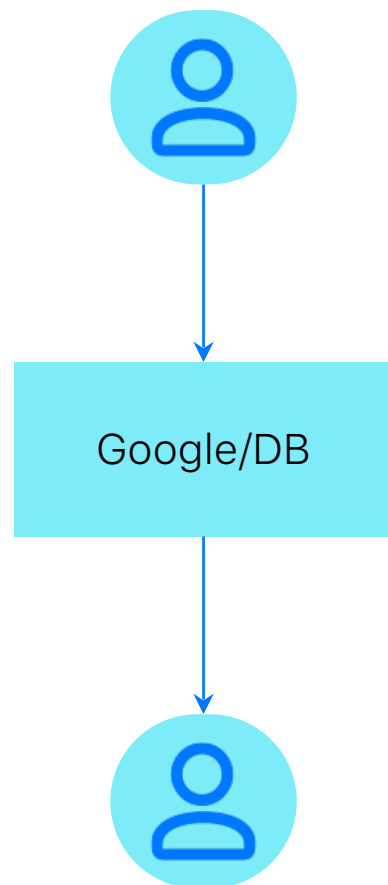


Recap

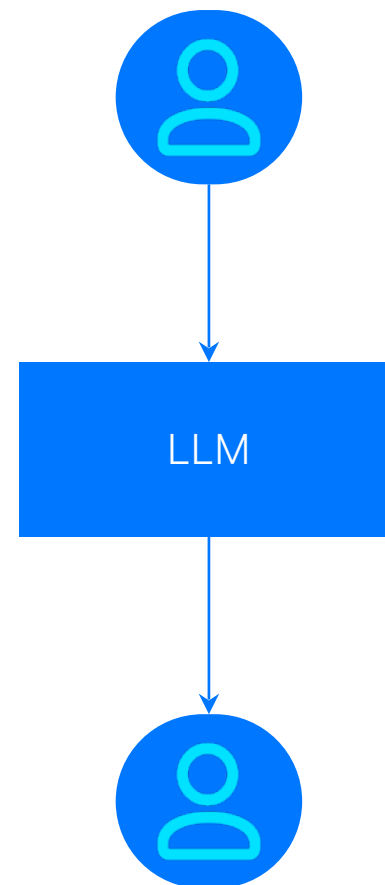
Что такое RAG

Retrieval vs Generation Search

- Результат поиска надо обработать перед использованием
- Контроль над использованием информации



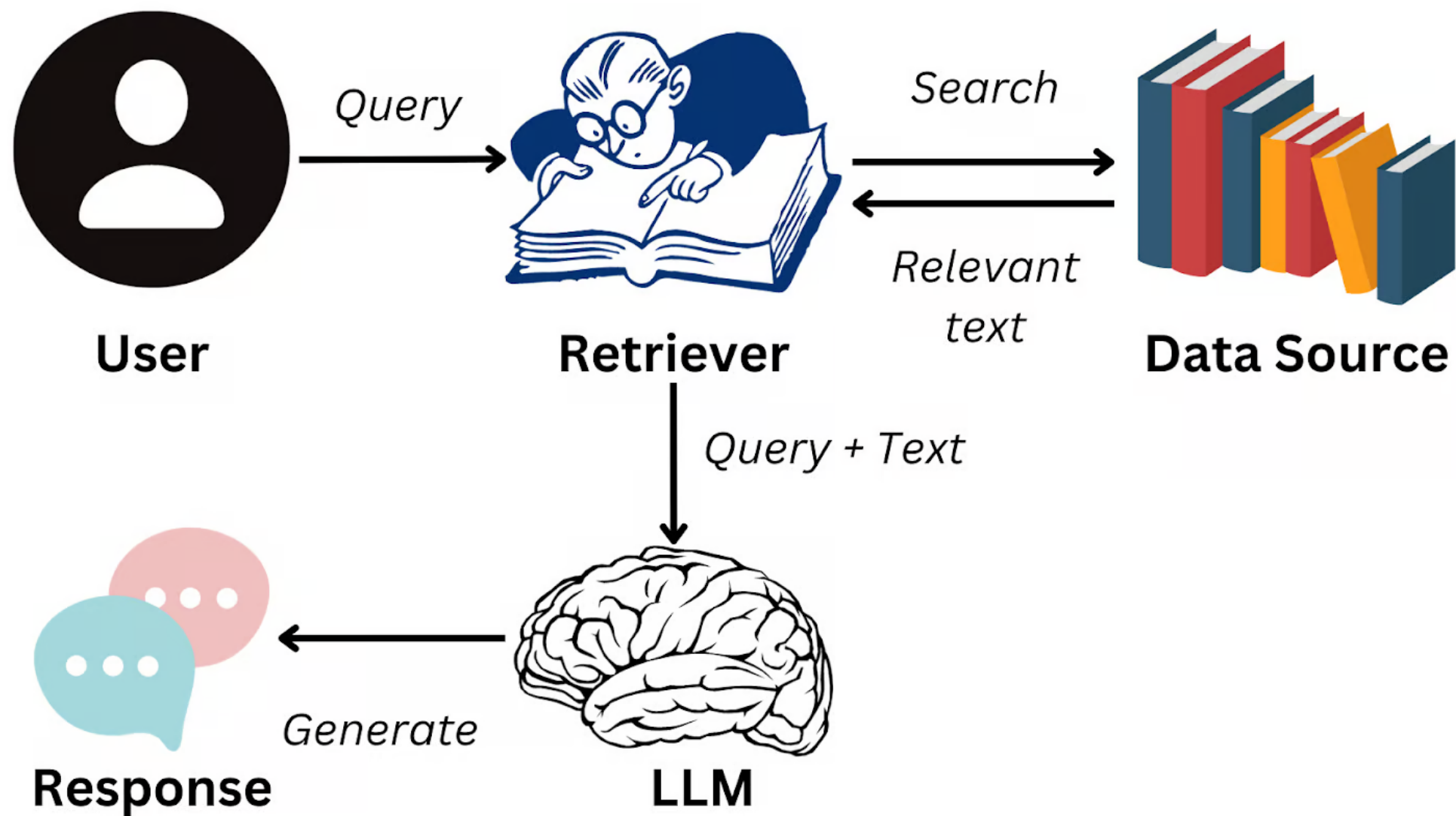
R+G



- Галлюцинации/нежелательное использование информации
- Результат поиска адаптирован к запросу

Retrieval Augmented Generation

- **Индексирование**
- Обработка запроса
- **Поиск**
- Подготовка для передачи LLM
- **Генерация**



Индексирование



Encoder

Подходят только энкодеры, которые обучались на похожие задачи

- Sentence Transformers (STS, ...)
- [mteb](#)
- Вряд ли будем дообучать
- Помним о длине контекста - нарежем текст на кусочки
- [Matryoshka Embeddings](#)



Indexing

Vectorstore	Delete by ID	Filtering	Search by Vector	Search with score	Async	Passes Standard Tests	Multi Tenancy	IDs in add Documents
AstraDBVectorStore	✓	✓	✓	✓	✓	✗	✗	✗
Chroma	✓	✓	✓	✓	✓	✗	✗	✗
Clickhouse	✓	✓	✗	✓	✗	✗	✗	✗
CouchbaseVectorStore	✓	✓	✗	✓	✓	✗	✗	✗
DatabricksVectorSearch	✓	✓	✓	✓	✓	✗	✗	✗
ElasticsearchStore	✓	✓	✓	✓	✓	✗	✗	✗
FAISS	✓	✓	✓	✓	✓	✗	✗	✗
InMemoryVectorStore	✓	✓	✗	✓	✓	✗	✗	✗

Эмбединги положим в векторную БД

- Которая поддерживает поиск по близости
- За вменяемое время
- Поддерживает добавление векторов (если собираетесь добавлять)

Поиск

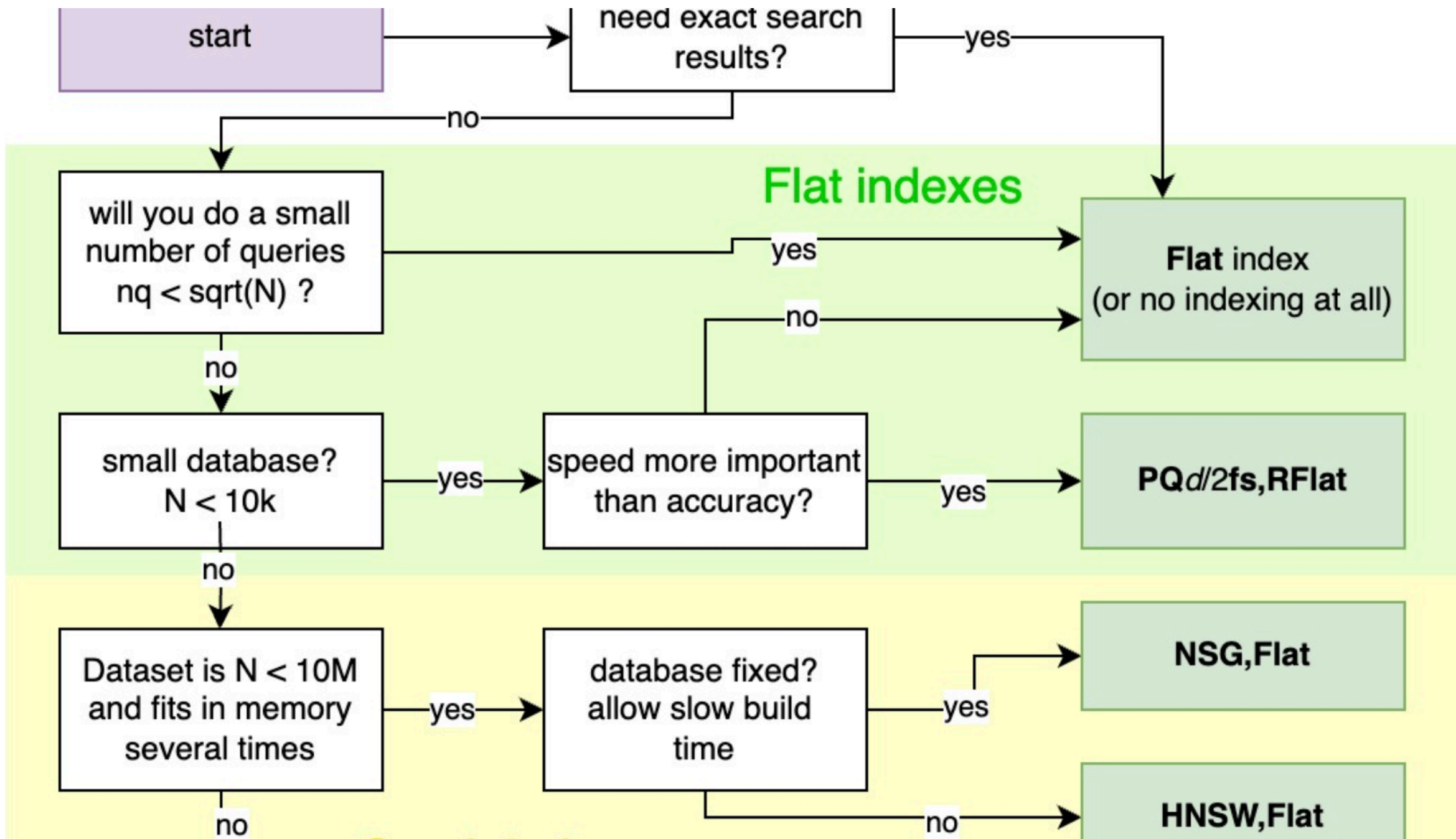


Approximate Nearest Neighbour Search



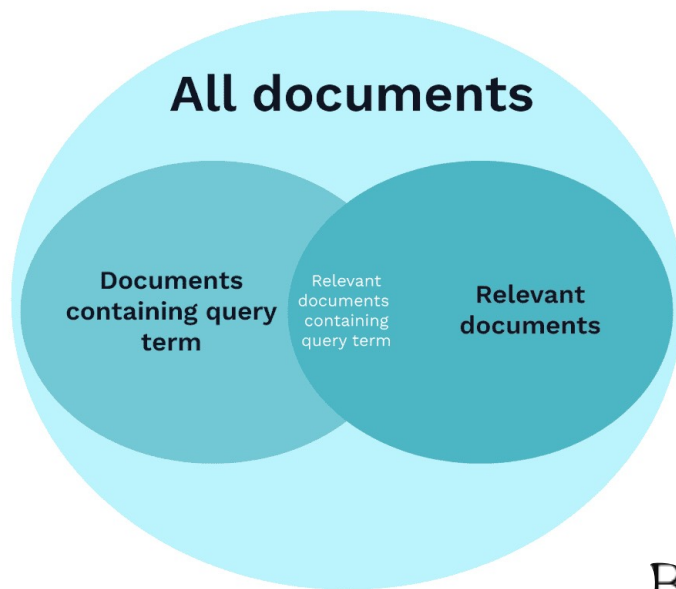
Будем искать ближайшие векторы
в некотором подмножестве

- FAISS (HNSW, LSH, PQ)
- Chroma
- ...



Lexical retrieval

Best Match 25



Улучшенный tf-idf

$$\text{BM25}(D, Q) = \sum_{t \in Q} \text{IDF}(t) \cdot \frac{f(t, D) \cdot (k_1 + 1)}{f(t, D) + k_1 \cdot (1 - b + b \cdot \frac{|D|}{\text{avg}|D|})}$$

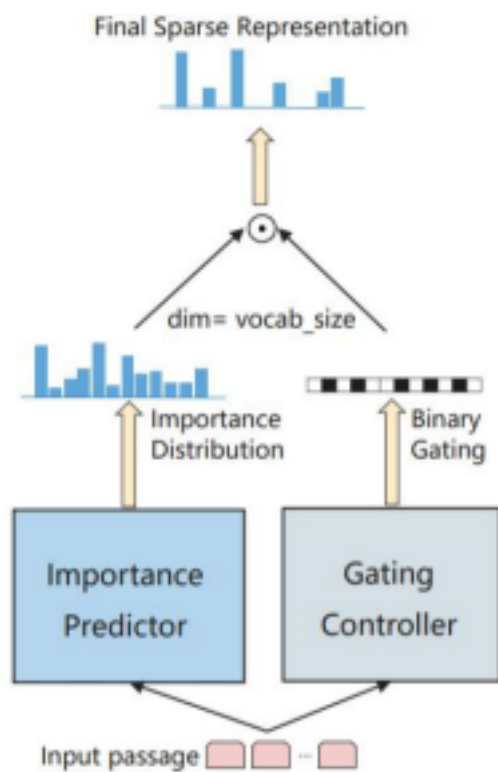
Combined retrieval



Как подружить лексический и семантический поиск

- Можно просто объединить кандидатов
- А можно учить энкодеры учитывать ключевые слова

Combined retrieval



(a) SparTerm Model

SPLADE: Sparse Lexical and Expansion Model for First Stage Ranking

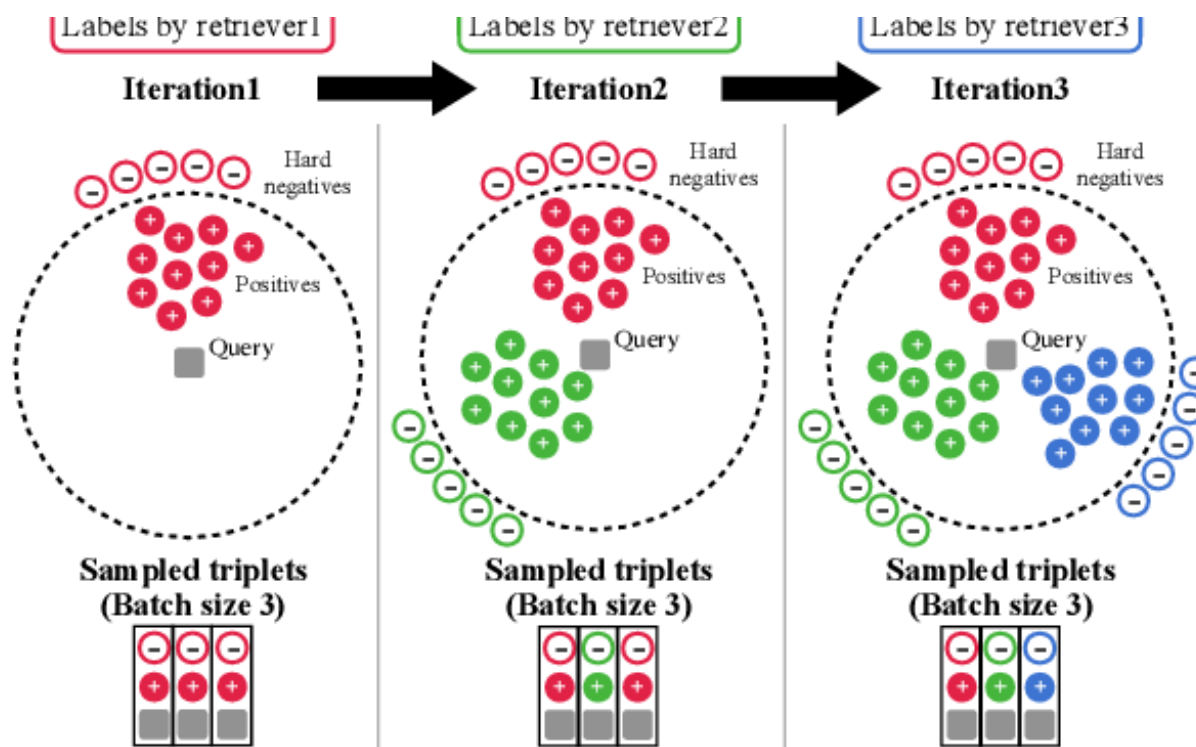
- combines keyword search and semantic search by creating sparse vectors based on vocabulary terms
- BERT's MLM head to compute term-level scores for tokens

<https://liuquncn.github.io/talks/20210602-BAAI-IR-Forum/SparTerm%20Learning%20Term-based%20Sparse%20Representation.pdf>

Combined retrieval

How to Train Your [DRAGON](#): Diverse Augmentation Towards Generalizable Dense Retrieval

- The model collects relevance labels from different types of retrievers: Sparse retrievers (e.g., BM25) ,Dense retrievers (e.g., DPR, ANCE)



Улучшения

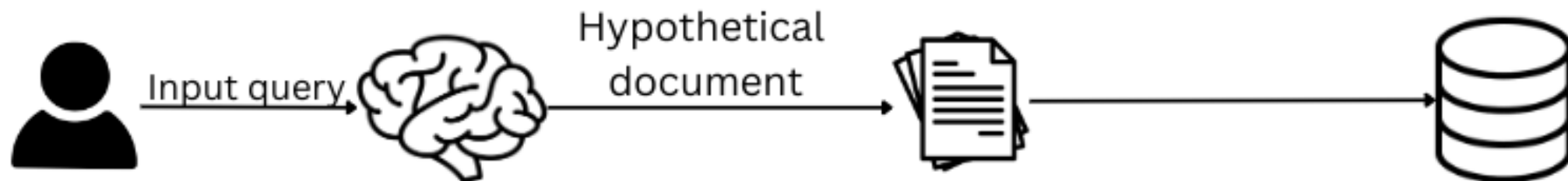


Query improvement

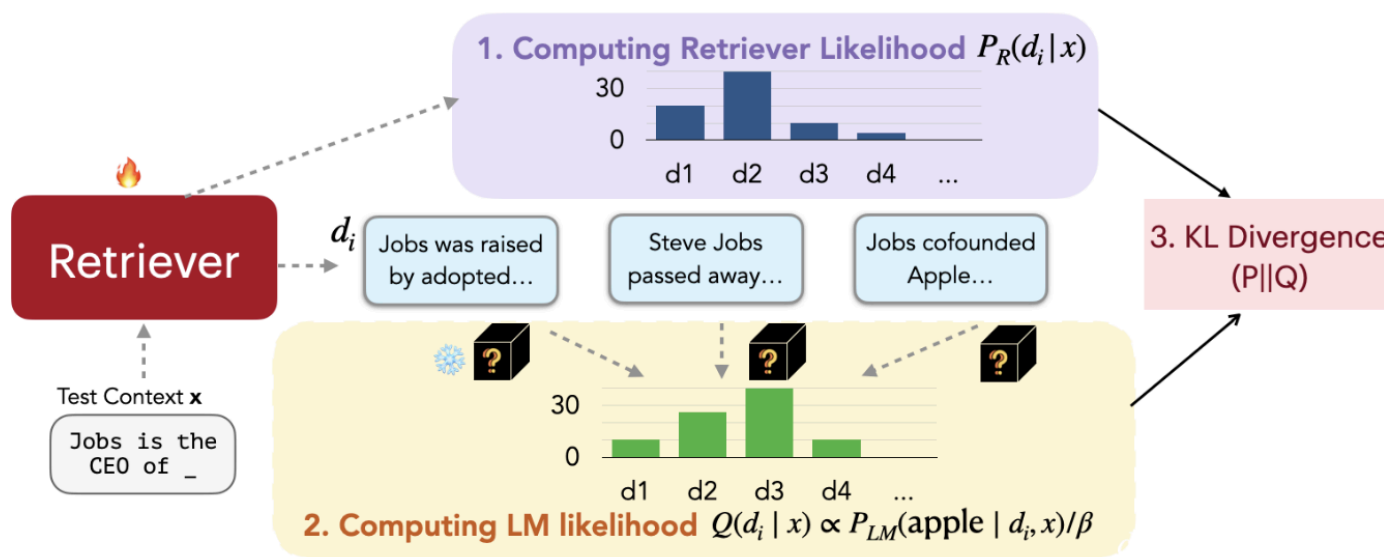
Будем искать что-то другое

- Query Encoder fine-tune
- Исправим запрос (spellchecker)
- Найдем соседей для аугментированного запроса
- Hypothetical Document Embeddings ([HyDE](#))

Perform retrieval
on the hypothetical document



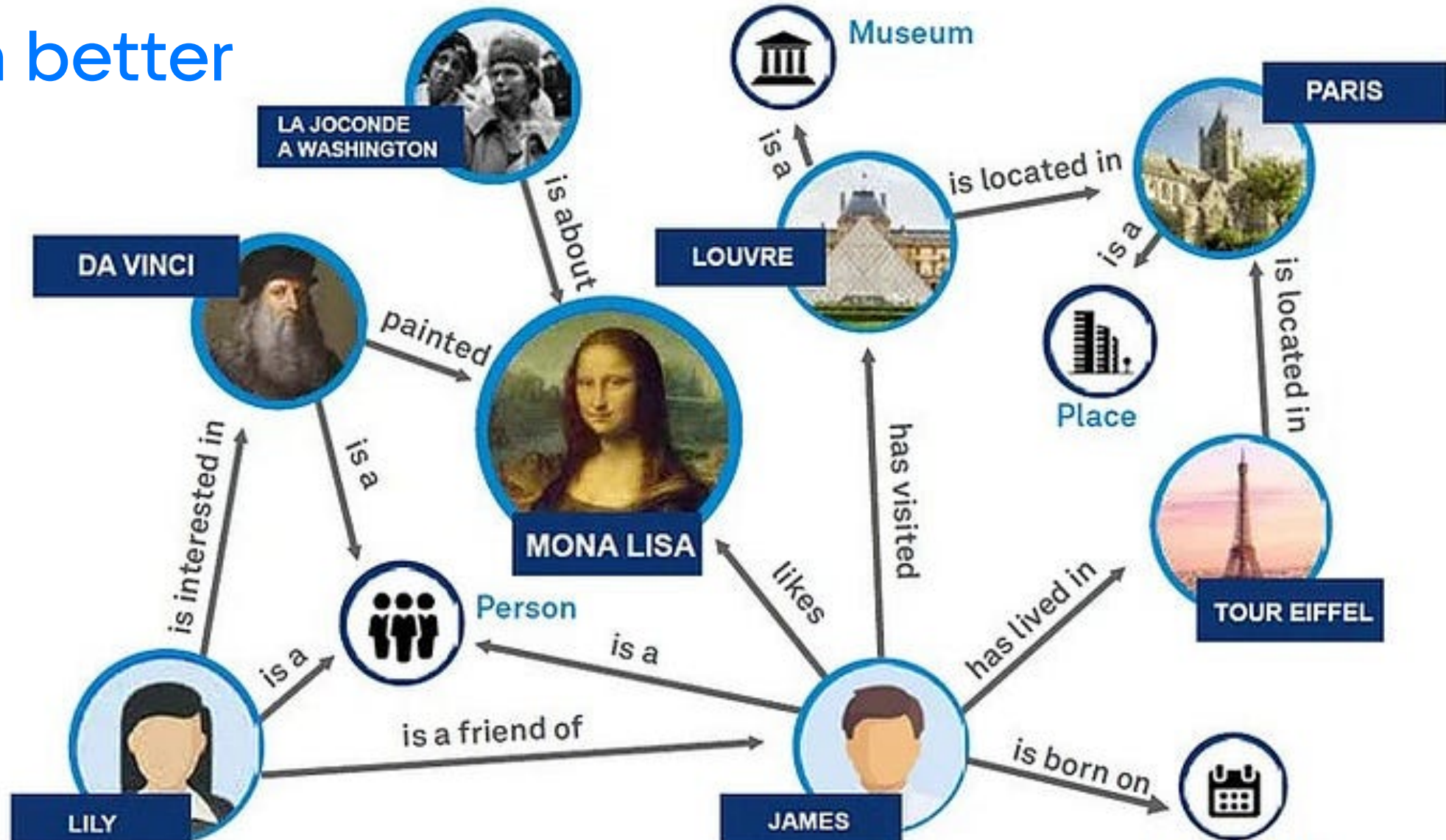
Retrieved candidates



Нашли ближайших соседей. Что дальше?

- Re-rank - вот здесь можно дообучать
- [REPLUG](#): Retrieval-Augmented Black-Box Language Models, In-Context RALM
- Cross-encoder
- Merge chunks

Even better



https://miro.medium.com/v2/resize:fit:700/0*rm_fRSPovV1wfTqH.jpg

Поиск по графу знаний

Building Knowledge Graphs in 10 steps



1 Clarify your business & expert requirements



2 Gather and analyze relevant data



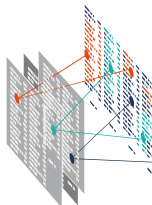
3 Clean data to ensure data quality



4 Create your semantic data model



5 Integrate data with ETL or virtualization



6 Harmonize data via reconciliation, fusion and alignment



7 Architect the data management and search layer



8 Augment your graph via reasoning, analytics and text analysis



9 Maximize the usability of your data



10 Make your KG easy to maintain and evolve

<https://www.ontotext.com/knowledgehub/fundamentals/how-to-building-knowledge-graphs-in-10-steps/>

Поиск по графу знаний

Индексация

- Entites
- Relationship
- Ontology
- RDF
- Align
- Graph Database

Поиск

- Lexical
- Semantic

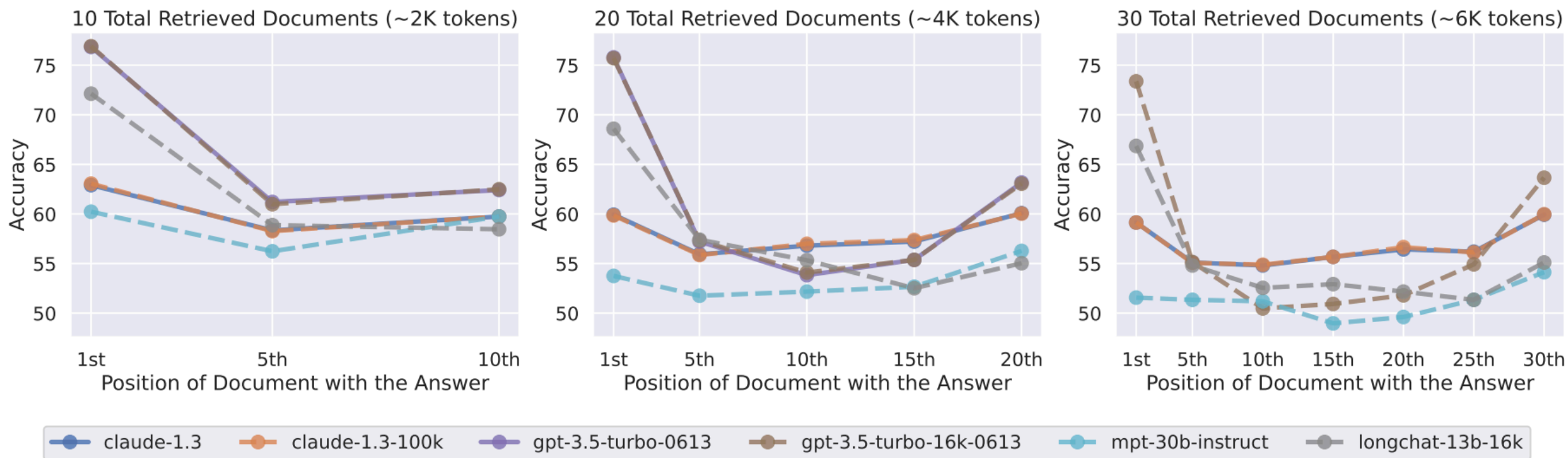
Генерация



Augmented Generation

In-context learning - просто добавим в контекст?

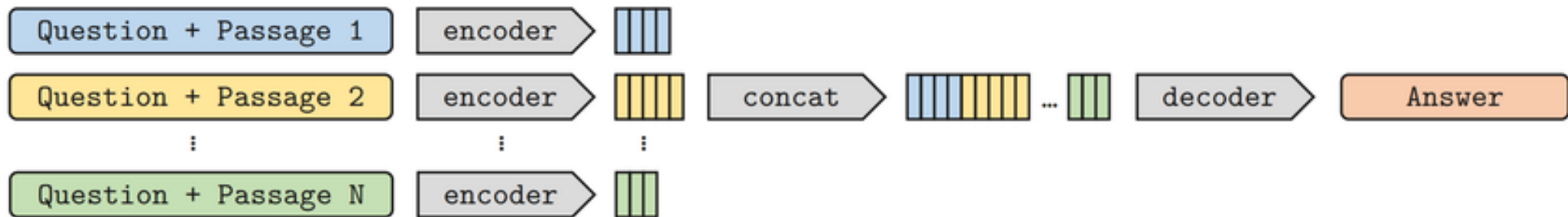
- [Lost in the Middle](#): How Language Models Use Long Contexts



Augmented Generation

Concatenate hidden states

- [FID](#): Leveraging Passage Retrieval with Generative Models for Open Domain Question Answering



Augmented Generation

Аугментируем на каждом шаге генерации

- [kNN-LM](#): Efficient Nearest Neighbor Language Models

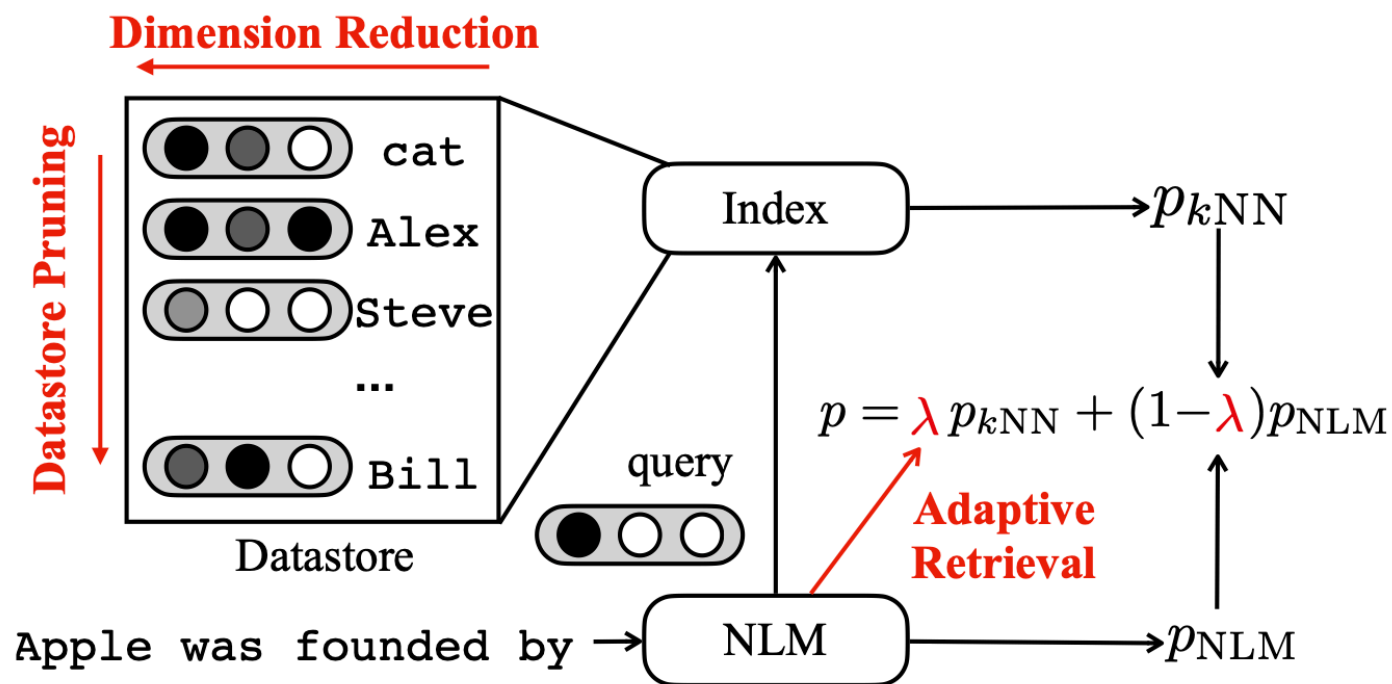
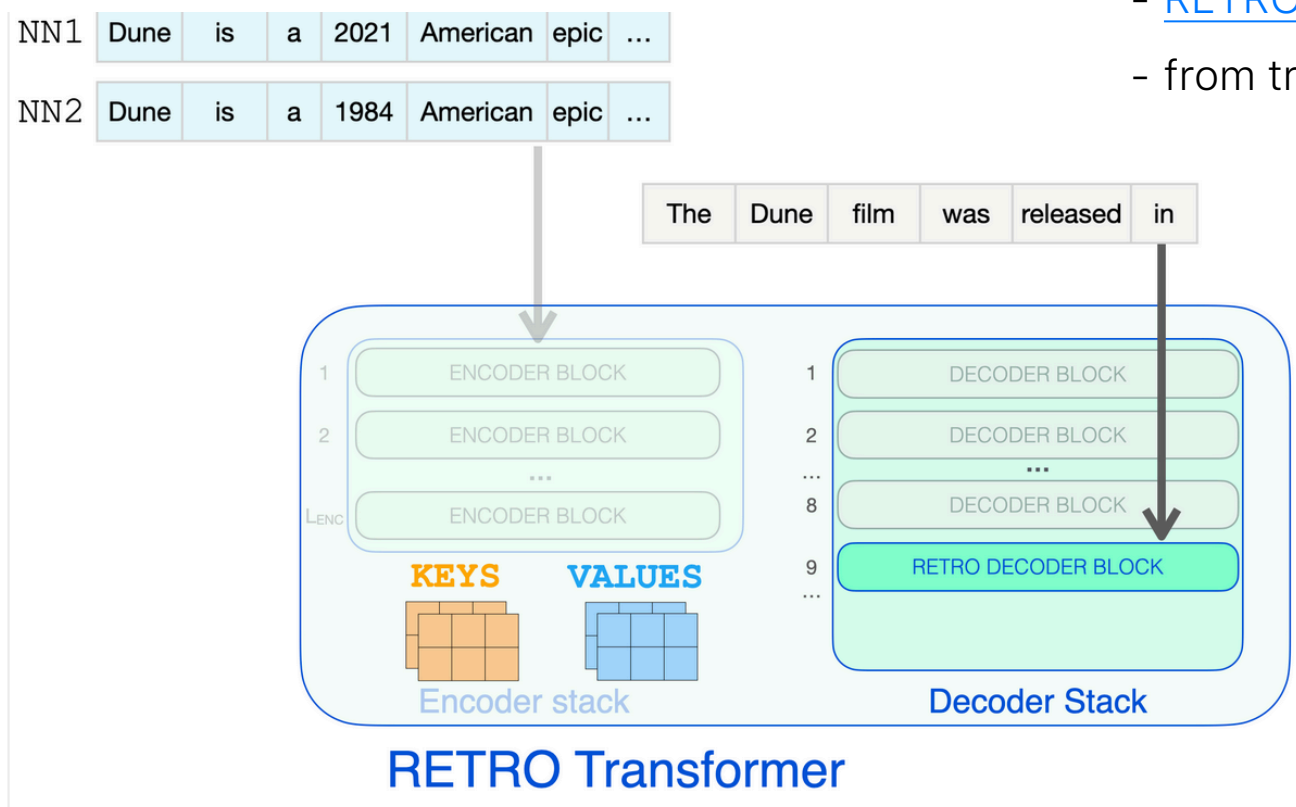


Figure 2: Illustration of k NN-LM . The datastore consists of paired context representations and the corresponding next tokens. The index represents a method that performs approximate k -nearest neighbor search over the datastore. Red and bolded text represent three dimensions that we explore in this paper to improve the efficiency. In adaptive retrieval, we use a predictive model to decide whether or not to query the datastore.

Augmented Generation

Сложно добавим с attn

- [RETRO](#): Improving language models by retrieving
- from trillions of tokens



Input prompt tokens RETRO Decoder Block to start information retrieval

<https://jalammar.github.io/illustrated-retrieval-transformer/>

Augmented Generation

Method	What is Retrieved?	Where is it Injected?
KNN-LM	Token embeddings	At output probability layer
Fusion-in-Decoder (FiD)	Full passages	Inside the decoder
RETRO	Small text chunks	Inside transformer layers
RAG	Full documents	Input to the generative model

Добавим целый документ

- RAG-Sequence, RAG-Token ([RAG](#) Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks)

На практике



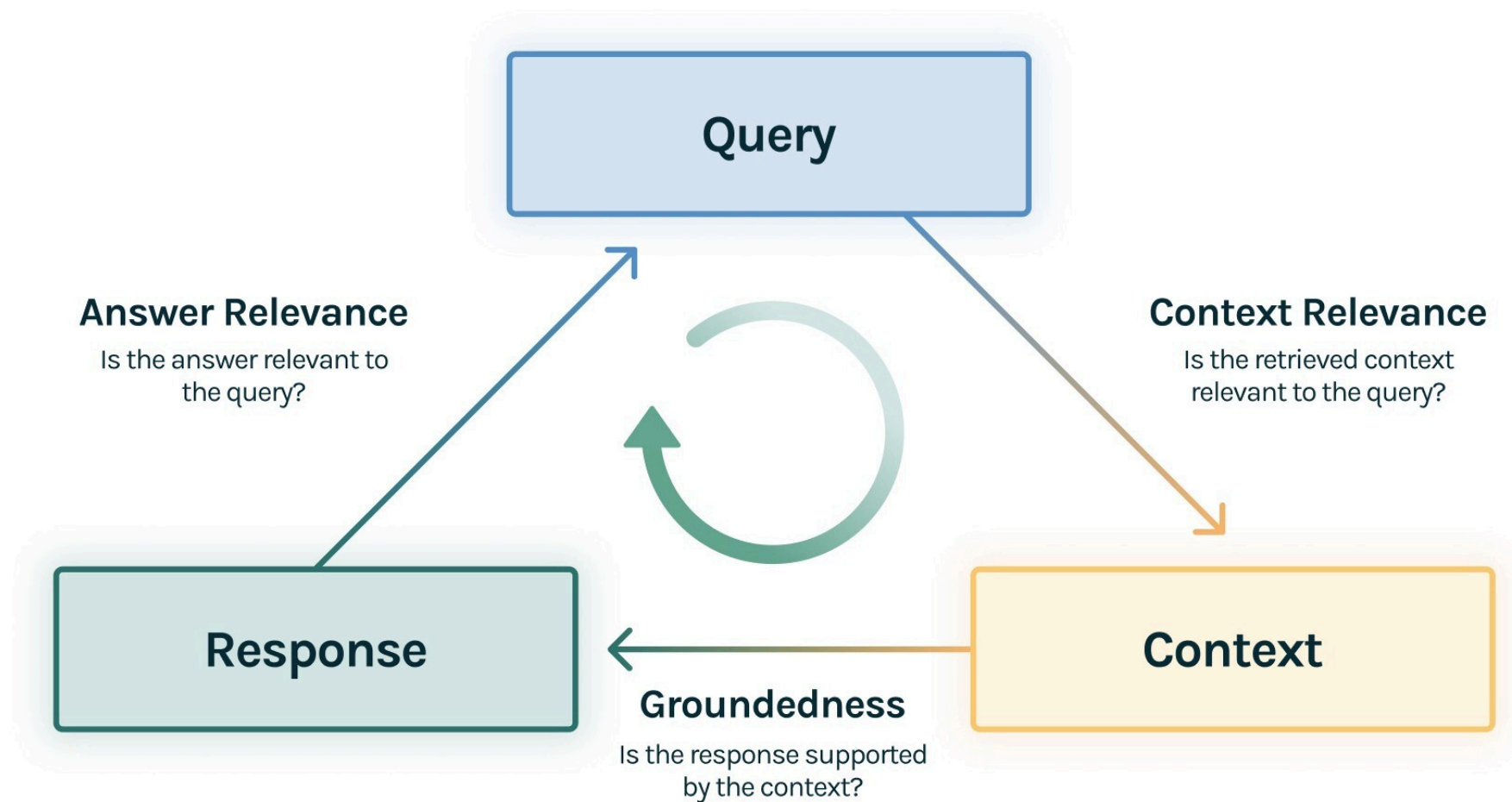
Evaluation



Оцениваем конечный ответ: RAG as RALM

- BLEU, ROUGE
- Perplexity
- Human Eval
- LLM as a judge

Оцениваем по частям



https://www.trulens.org/getting_started/core_concepts/rag_triad/

Let's launch something



[картинка](#)



Спасибо
за внимание!